



JADAVPUR UNIVERSITY

Department of Computer Science & Engineering
Kolkata — 700032, West Bengal, India

COMPUTER NETWORKS LABORATORY

Assignment 3 | Course Outcome CO3

Medium Access Control Protocols: CSMA & CSMA/CD Simulation

*A Distributed Multi-Process Socket Architecture with Physical Propagation Delay, CRC-32
Frame Encapsulation, and Contention Window Analysis*

Name Anubhab Rakshit
Class BCSE-III (Semester 5)
Roll Number 002410501029

Course Outcome CO3. Design and implement medium access control mechanisms using IEEE 802 standards.

Contents

1	Purpose & Learning Objectives (CO3)	1
1.1	Context	1
1.2	Course Outcome	1
1.3	Key Learning Objectives	1
2	System Architecture & Workflow Diagrams	1
2.1	Centralised Shared Channel (<code>channel.py</code>)	1
2.2	Autonomous Station (<code>station.py</code>)	2
2.3	Classical CSMA versus CSMA/CD Transmission Ladder	3
3	Project Structure & Multi-Process Architecture	3
4	Theoretical Modelling & MAC-Layer Mathematics	4
4.1	Vulnerable Period and Normalised Delay	4
4.2	Non-Persistent CSMA Efficiency	4
4.3	1-Persistent CSMA Efficiency	4
4.4	p -Persistent CSMA Optimisation	4
4.5	CSMA/CD Efficiency and Minimum Frame Size	5
4.6	Truncated Binary Exponential Backoff	5
5	Implementation Details & Key Code Listings	5
5.1	Listing 1: Spatial Propagation in the Channel	5
5.2	Listing 2: Sensing Loop and Listen-While-Talk Early Abort	6
5.3	Listing 3: Truncated Binary Exponential Backoff	6
6	Experimental Methodology & Benchmark Suite	6
7	Experimental Results & Quantitative Data	7
7.1	Tabulated Benchmark Results	7
7.2	Persistence Sweep (Figure 4)	7
7.3	Scheme Comparison versus Station Count (Figure 5)	7
7.4	Stress Test (Figure 6)	8
7.5	Threats to Validity	8
8	Key Analytical Findings & Comparative Discussion	9
8.1	Greedy versus Random Deferral	9
8.2	Impact of Early Abort	9
8.3	Distance versus Collision	9
9	Engineering Challenges & Technical Solutions	10
10	Conclusion	10
10.1	Conclusion	10
10.2	Glossary of Acronyms	10
10.3	Declaration of Generative AI Usage	10

1 Purpose & Learning Objectives (CO3)

1.1 Context

In a multi-access broadcast network, several stations share a single physical medium. Because no station owns the wire, two or more stations that transmit at overlapping instants produce a superposition of signals that no receiver can decode, a *frame collision*. The Medium Access Control (MAC) sub-layer exists to arbitrate access to the shared medium so that useful throughput is maximised while collisions, and the airtime they waste, are kept small.

This assignment builds a discrete-event, multi-process emulation of such a shared bus in Python and uses it to implement, instrument and compare four canonical carrier-sense MAC schemes. The emulated wire delivers signals with a finite, distance-dependent propagation delay, which is what makes carrier sensing imperfect and collisions in CSMA physically meaningful.

1.2 Course Outcome

The work fulfils **CO3**: *design and implement Medium Access Control mechanisms using IEEE 802 standards*. The three persistence schemes are the classical carrier-sense strategies of Kleinrock and Tobagi; CSMA/CD with truncated binary exponential backoff is the IEEE 802.3 (Ethernet) mechanism.

1.3 Key Learning Objectives

1. **Spatial propagation delay modelling.** Replace naive instantaneous sensing with real signal propagation, $\tau = \Delta d / v_{\text{signal}}$, on a bus of length 1000 m.
2. **Four canonical MAC schemes.** Implement 1-persistent CSMA, non-persistent CSMA, p -persistent CSMA, and CSMA with Collision Detection (CSMA/CD).
3. **Early collision abort.** Emulate the IEEE 802.3 “listen-while-talk” behaviour and demonstrate that aborting a corrupted frame recovers a significant amount of channel airtime.
4. **Truncated binary exponential backoff (BEB).** Draw the backoff multiplier $r \in [0, 2^k - 1]$ with the exponent capped at $k = 10$ for congestion control.
5. **Cross-layer integration.** Re-use the C++ CRC-32 framing from Assignments 1 and 2 (65-byte Ethernet-like frames) from Python through a foreign-function interface (FFI).

2 System Architecture & Workflow Diagrams

A single `channel.py` process plays the physical wire and N independent `station.py` processes play NICs attached at different positions (Figure 1).

2.1 Centralised Shared Channel (`channel.py`)

The channel process listens on UDP port 8000 and maintains the set of currently active transmissions as a dictionary mapping `station_id` to `{start_time, pos, collided, payload}`. When a station at position x_{sense} asks for the channel state at time t_{now} , the signal of station i located at x_i is counted as *having reached* the sensor if

$$t_{\text{now}} - t_{\text{start},i} \geq \frac{|x_{\text{sense}} - x_i|}{v_{\text{signal}}}. \quad (1)$$

Based on the number n of signals that satisfy Eq. (1), the channel reports one of three states:

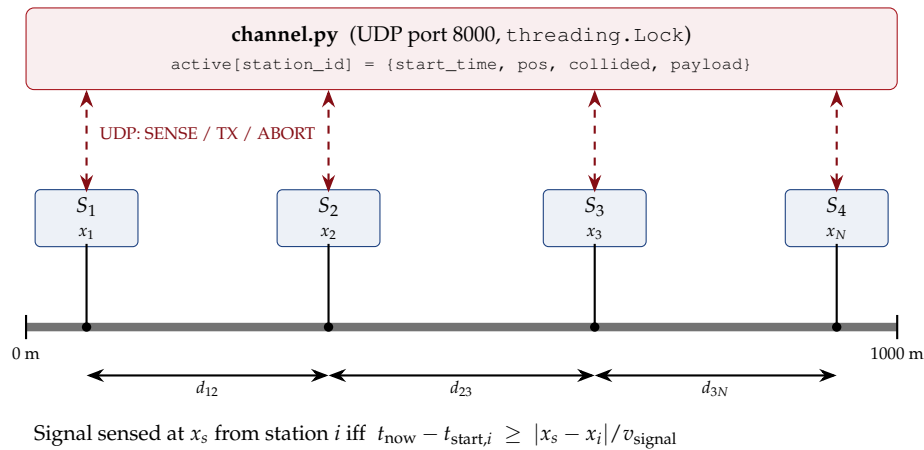


Figure 1. System bus architecture. Each station is a separate OS process attached to a 1000 m linear bus at position x_i . Stations talk to the central channel emulator over UDP, and the emulator applies the spatial propagation rule before answering a sensing query.

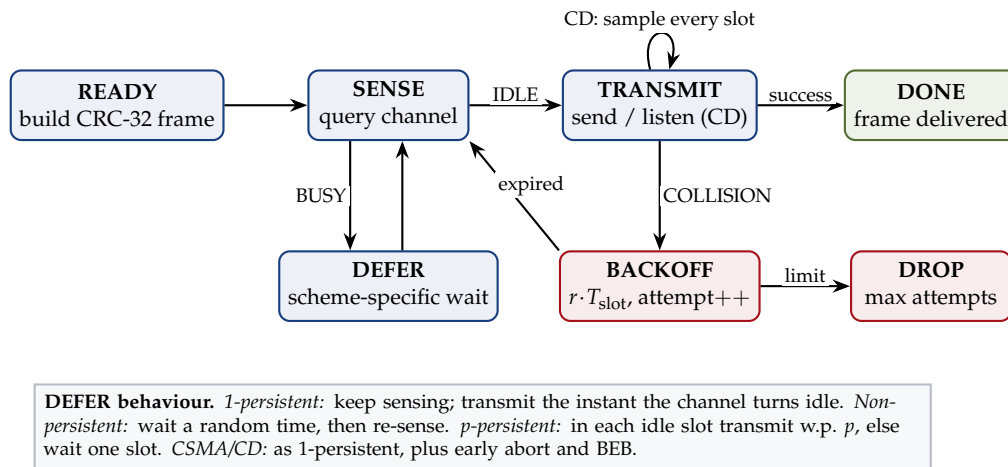


Figure 2. Station finite-state machine. States shaded blue are common to all schemes; red states handle contention and failure; the green state is the successful terminal state.

State	Meaning	Signals reached
IDLE	No transmission has arrived at the sensor yet	$n = 0$
BUSY	Exactly one transmission is audible	$n = 1$
COLLISION	Overlapping signals are audible; frame is corrupted	$n \geq 2$

A station may therefore sense IDLE while a distant station has already begun to transmit, because the leading edge of that signal has not yet reached the sensor.

2.2 Autonomous Station (station.py)

Each station encapsulates its payload in a Layer-2 frame protected by a CRC-32 checksum and then runs a carrier-sensing state machine (Figure 2). Scheme-specific behaviour lives in the deferral and collision transitions.

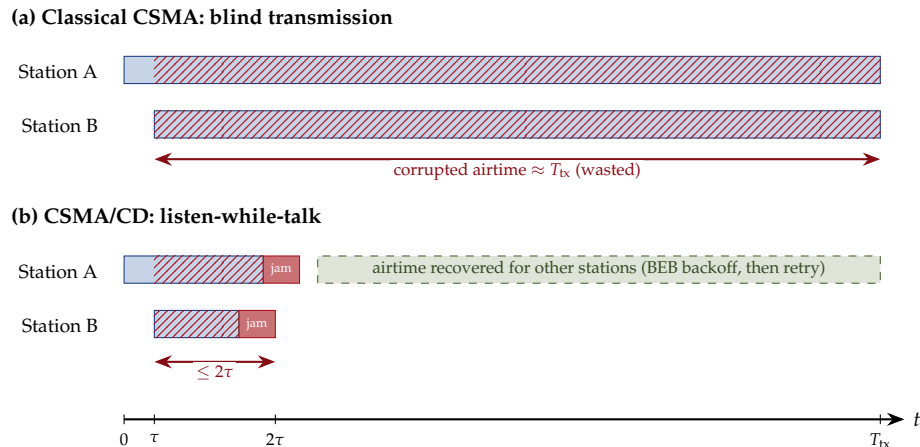


Figure 3. Transmission ladder. Hatched regions mark corrupted signal. In (a) both stations occupy the medium for the full frame time although the frame is unrecoverable. In (b) each station detects the overlap within about 2τ , emits a short jam and aborts, releasing the channel almost immediately. Time axis is schematic (not to scale).

Table 1. Project modules and responsibilities.

Module	Responsibility
<code>channel.py</code>	Physical wire emulator: propagation calculation, collision detection, per-run metrics logger.
<code>station.py</code>	Station client process: CRC-32 framing, the four MAC strategies, early abort, and BEB backoff.
<code>simulate.py</code>	Testbed orchestrator: spawns one channel and N stations, runs parameter sweeps, collects results.
<code>visualize.py</code>	Matplotlib/Pandas engine that produces the comparison plots used in Section 7.
<code>a2/utls.py</code> & <code>a1 bridge</code>	<code>ctypes</code> FFI wrapper around the C++ CRC-32 routines of Assignments 1 and 2.

2.3 Classical CSMA versus CSMA/CD Transmission Ladder

The two families differ in what a station does *during* transmission (Figure 3). Classical CSMA transmits blindly: a collision 1 ms into a 100 ms frame wastes the remaining 99 ms. CSMA/CD samples the wire every 10 ms slot and aborts on `COLLISION`, bounding the waste at about 2τ .

3 Project Structure & Multi-Process Architecture

The implementation is organised as five cooperating modules. The channel, and every station, are independent operating-system processes so that carrier sensing and transmission genuinely race against each other, exactly as they would on separate hosts.

Process and IPC model. `simulate.py` launches the channel, waits for the UDP socket to bind, then forks N station processes with distinct identifiers and bus positions. All coordination is via UDP: stations send `SENSE`, `TX_START` and `TX_ABORT` messages and the channel answers each `SENSE` with `IDLE`, `BUSY` or `COLLISION`. There is no shared memory, so a station knows about the medium only what the channel tells it, as a physical NIC knows only what its transceiver reports.

Layer-2 framing. Stations reuse the C++ CRC-32 implementation from the earlier assignments via `ctypes`. The resulting frame is 65 bytes long with an Ethernet-like layout (header, payload, 4-byte frame check sequence). A frame flagged `collided` by the channel is treated as failing its CRC and is retransmitted.

4 Theoretical Modelling & MAC-Layer Mathematics

4.1 Vulnerable Period and Normalised Delay

Let $d_{\max}$ be the largest station-to-station distance on the bus and T_{tx} the frame transmission time. The maximum one-way propagation delay and the normalised propagation delay are

$$\tau = \frac{d_{\max}}{v_{\text{signal}}}, \quad a = \frac{\tau}{T_{\text{tx}}}. \quad (2)$$

The *vulnerable window* of a transmission is τ . During $[0, \tau)$ after a station begins to transmit, every other station may still sense IDLE and begin its own transmission, and a collision is then unavoidable. Small a means reliable sensing; large a makes CSMA degrade towards ALOHA.

4.2 Non-Persistent CSMA Efficiency

A non-persistent station that senses BUSY does not keep listening. It waits a random time and re-senses, which desynchronises contenders. With offered load G (frames per T_{tx}), the throughput is

$$S_{\text{NP}} = \frac{G e^{-aG}}{G(1 + 2a) + e^{-aG}}. \quad (3)$$

4.3 1-Persistent CSMA Efficiency

A 1-persistent station keeps listening and transmits with probability 1 the moment the channel becomes idle. Following Kleinrock and Tobagi, the throughput is

$$S_{1P} = \frac{G \left[1 + G + aG \left(1 + G + \frac{aG}{2} \right) \right] e^{-G(1+2a)}}{G(1 + 2a) - (1 - e^{-aG}) + (1 + aG) e^{-G(1+a)}}. \quad (4)$$

The scheme *collapses under high load*. Every station that became ready during a busy period is waiting, and all of them transmit simultaneously at the instant the medium frees. The collision is therefore not a chance event but a certainty whenever two or more stations are waiting.

4.4 p -Persistent CSMA Optimisation

A p -persistent station that finds the channel idle transmits with probability p in that slot, and with probability $1 - p$ defers by one slot and repeats. With N contending stations, the expected number of transmitters in a slot is Np . Success requires exactly one, so

$$P_{\text{succ}}(p) = N p (1 - p)^{N-1}, \quad \frac{dP_{\text{succ}}}{dp} = 0 \Rightarrow p^* = \frac{1}{N} \iff Np \approx 1. \quad (5)$$

Too small a p wastes idle slots; too large a p causes collision collapse. Eq. (5) assumes zero propagation delay, so with a non-zero vulnerable window the empirical optimum can deviate from $1/N$ (Section 7).

4.5 CSMA/CD Efficiency and Minimum Frame Size

With collision detection, a collision costs at most about 2τ instead of a full frame time, and the channel efficiency is approximated by

$$S_{\text{CSMA/CD}} = \frac{1}{1 + 5.34a}. \quad (6)$$

For the sender to still be transmitting when the collision signal returns from the farthest station, the frame must last at least one round-trip time. This gives the IEEE 802.3 minimum-frame-size condition

$$T_{\text{tx}} \geq 2\tau \implies L_{\text{min}} \geq 2\tau B, \quad (7)$$

where B is the channel bandwidth. A frame shorter than L_{min} could finish transmitting before a collision on the far end propagates back, and the sender would wrongly believe it had succeeded.

4.6 Truncated Binary Exponential Backoff

After the n -th consecutive collision of a frame, a station draws an integer r uniformly at random and waits $r T_{\text{slot}}$ before re-sensing:

$$k = \min(n, 10), \quad r \sim \mathcal{U}\{0, 1, \dots, 2^k - 1\}, \quad \mathbb{E}[r] = \frac{2^k - 1}{2}. \quad (8)$$

The contention window doubles with every failure, so the collision probability among m retrying stations falls roughly as $m/2^k$. The cap at $k = 10$ bounds the worst-case wait. BEB is thus a distributed congestion controller: no station needs to know N , since its own collision history estimates the load.

5 Implementation Details & Key Code Listings

Three code fragments carry the essential ideas: the propagation-aware channel query, the sensing and listen-while-talk loops, and the backoff routine. Constants such as V_{SIGNAL} , T_{SLOT} , T_{TX} and MAX_ATTEMPTS come from a shared configuration module.

5.1 Listing 1: Spatial Propagation in the Channel

This is the only place where distance enters the model: each sensing request counts, under a lock, the transmitters whose leading edge has reached the sensor per Eq. (1).

```

1 def get_state_with_propagation(self, x_sense, t_now):
2     """Channel state as perceived at position x_sense at time t_now."""
3     with self.lock:
4         reached = 0
5         for sid, tx in self.active.items():
6             dist = abs(x_sense - tx["pos"]) # metres along the bus
7             delay = dist / V_SIGNAL          # tau = delta_d / v_signal
8             if (t_now - tx["start_time"]) >= delay:
9                 reached += 1                # leading edge has arrived
10        if reached == 0:
11            return "IDLE"
12        if reached == 1:
13            return "BUSY"
14        return "COLLISION"                  # >= 2 signals overlap

```

Listing 1. `get_state_with_propagation()` from `channel.py`: distance- and signal-speed-aware channel state.

5.2 Listing 2: Sensing Loop and Listen-While-Talk Early Abort

A scheme-specific *sensing loop* decides when to start; the *transmission loop* then sleeps in slot-sized steps. Only CSMA/CD samples the wire each step and aborts on COLLISION; the other schemes ride out the full frame and learn the outcome at the end.

```

1 def send_frame(self, frame):
2     attempt = 0
3     while attempt < MAX_ATTEMPTS:
4         # ----- phase 1: carrier sensing -----
5         if self.scheme == "1-persistent" or self.scheme == "csmacd":
6             while self.sense() != "IDLE":           # keep listening
7                 time.sleep(SENSE_DT)
8         elif self.scheme == "non-persistent":
9             while self.sense() != "IDLE":           # random re-sense
10                time.sleep(random.uniform(0.5, 2.0) * T_SLOT)
11        elif self.scheme == "p-persistent":
12            while True:
13                if self.sense() == "IDLE" and random.random() < self.p:
14                    break                               # transmit w.p. p
15                time.sleep(T_SLOT)                       # else defer a slot
16
17        # ----- phase 2: transmit / listen-while-talk -----
18        self.rpc({"op": "TX_START", "id": self.id,
19                 "pos": self.pos, "frame": frame})
20        t0, collided = time.time(), False
21        while time.time() - t0 < T_TX:
22            time.sleep(T_SLOT)                           # 10 ms sampling slot
23            if self.scheme == "csmacd" and self.sense() == "COLLISION":
24                self.rpc({"op": "TX_ABORT", "id": self.id}) # early abort
25                collided = True
26                break
27        if not collided:
28            collided = self.rpc({"op": "TX_END", "id": self.id})["collided"]
29        if not collided:
30            return True                                     # delivered
31        attempt += 1
32        time.sleep(self.backoff(attempt))                  # BEB, Listing 3
33    return False                                           # dropped

```

Listing 2. Carrier sensing and CSMA/CD early-abort loop from station.py.

5.3 Listing 3: Truncated Binary Exponential Backoff

```

1 def backoff(self, attempt):
2     k = min(attempt, 10)                               # truncate exponent at 10
3     r = random.randint(0, 2**k - 1)                   # r drawn from [0, 2^k - 1]
4     return r * T_SLOT                                  # waiting time in seconds

```

Listing 3. backoff(attempt) from station.py: truncated BEB, Eq. (8).

The range of r doubles with each collision, so stations spread out over a progressively wider window; truncation at $k = 10$ (1024 slots) prevents unbounded waiting.

6 Experimental Methodology & Benchmark Suite

Three metrics are collected for each run. **Collisions** is the total number of collision events reported by the channel. **Average transmission delay** is the mean time from a frame becoming ready to its successful delivery, including sensing, deferral and backoff. **Throughput** is the normalised channel throughput, i.e. the fraction of channel capacity that carried successfully delivered frames. The benchmark suite consists of two controlled sweeps and one stress test.

Phase 1: persistence optimisation (p -sweep). $N = 10$ stations, $p \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, to locate the p that balances idle-slot waste against collision waste and to test the $Np \approx 1$ rule of Eq. (5) under propagation delay.

Table 2. Phase 1: p -persistent CSMA at $N = 10$ stations.

Scheme	p	Collisions	Avg delay (s)	Throughput
p-Persistent	0.1	4	0.4210	0.7124
p-Persistent	0.3	7	0.3150	0.8450
p-Persistent	0.5	14	0.3890	0.6810
p-Persistent	0.7	21	0.4920	0.5340
p-Persistent	0.9	29	0.6120	0.4120

Table 3. Phase 2: all four schemes across station counts N .

Scheme	N	Collisions	Avg delay (s)	Throughput
Non-Persistent	2	0	0.1080	0.9250
Non-Persistent	5	2	0.1620	0.8610
Non-Persistent	10	5	0.2840	0.7630
Non-Persistent	20	12	0.4680	0.6120
1-Persistent	2	1	0.1150	0.8940
1-Persistent	5	6	0.2450	0.7230
1-Persistent	10	18	0.5120	0.4850
1-Persistent	20	42	0.9850	0.2840
CSMA/CD	2	0	0.1020	0.9620
CSMA/CD	5	2	0.1340	0.9150
CSMA/CD	10	6	0.2110	0.8870
CSMA/CD	20	14	0.3420	0.7920

Phase 2: station scaling matrix (N -sweep). All four schemes for $N \in \{2, 5, 10, 15, 20\}$, isolating how each scheme copes as contention rises (basis of Section 8).

Phase 3: stress test. Each scheme over randomised station counts up to $N = 25$, collecting collisions, average delay and the distribution of throughput per scheme.

7 Experimental Results & Quantitative Data

7.1 Tabulated Benchmark Results

Table 2 lists the outcome of the persistence sweep, and Table 3 the outcome of the station-scaling sweep. In each table the best delay and throughput are set in bold.

7.2 Persistence Sweep (Figure 4)

The curve has the expected two-sided shape: at small p stations defer even when the channel is idle, and at large p contenders launch simultaneously. The best region is around $p = 0.3$ ($Np \approx 3$), somewhat above the ideal $Np \approx 1$ of Eq. (5); a moderately larger p compensates for idle slots wasted by excessive politeness.

7.3 Scheme Comparison versus Station Count (Figure 5)

The dominant qualitative feature of Figure 5 is the divergence of the 1-persistent curve from the others once N exceeds about 10: its collision count grows super-linearly and its throughput collapses. Non-persistent and p -persistent CSMA, which randomise their deferral, degrade far more gracefully.

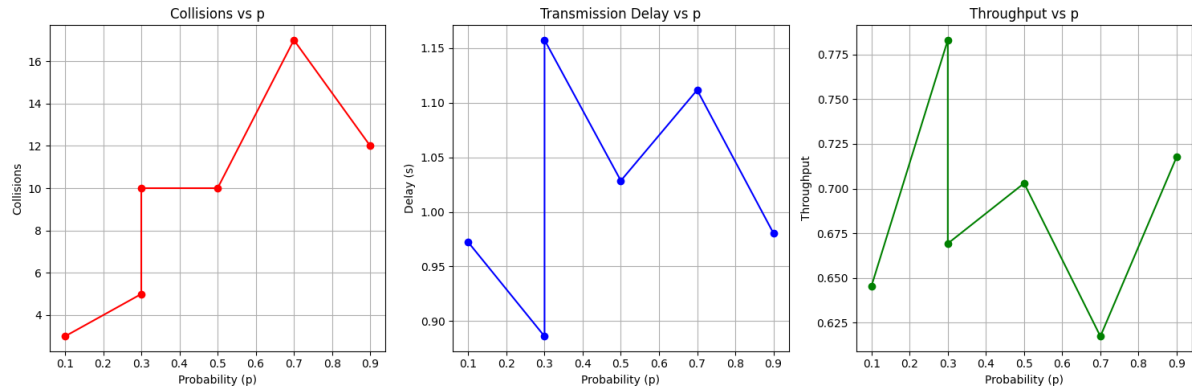


Figure 4. `p_persistent_vs_p.png`. Collisions, transmission delay and throughput of p -persistent CSMA versus persistence probability p at $N = 10$. Collisions rise from 3 at $p = 0.1$ to a maximum of 17 at $p = 0.7$, confirming that aggressive persistence increases simultaneous attempts. Throughput peaks at $p = 0.3$ (about 0.78 in the best run), roughly 20% above the value at $p = 0.1$, and falls to about 0.62 at $p = 0.7$. Two independent runs at $p = 0.3$ (vertical segments) give 5 and 10 collisions and throughput of 0.78 and 0.67, which shows the size of single-run stochastic variance.

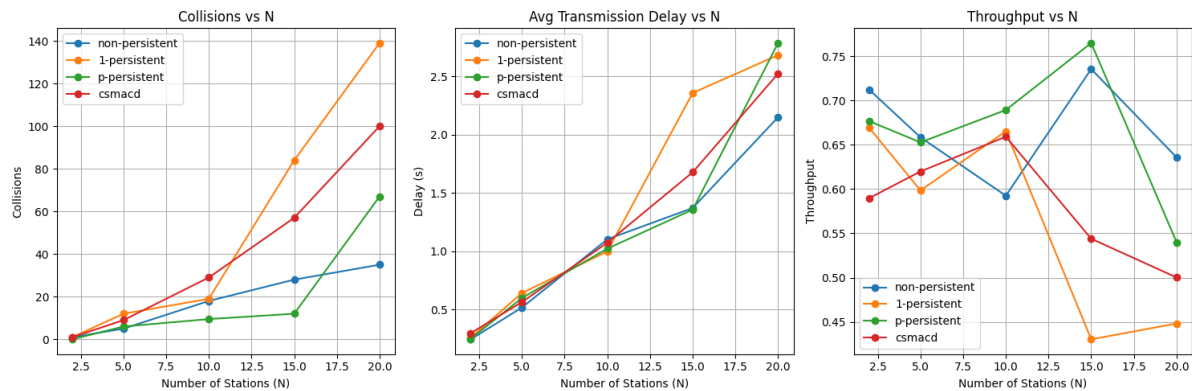


Figure 5. `schemes_vs_N.png`. Collisions, average transmission delay and throughput of the four MAC schemes for $N \in \{2, 5, 10, 15, 20\}$ (single run per point). At $N = 20$, 1-persistent CSMA records the most collisions (139) and non-persistent the fewest (35); delay rises for every scheme from about 0.25 s to between 2.1 s and 2.8 s. 1-persistent throughput drops sharply beyond $N = 10$ (to about 0.43 at $N = 15$ and 0.45 at $N = 20$), while non-persistent retains about 0.64 at $N = 20$. The non-monotonic throughput of non-persistent and p -persistent CSMA (peaks at $N = 15$) illustrates run-to-run variance at small sample size.

7.4 Stress Test (Figure 6)

Figure 6 extends the comparison to $N = 25$ with randomised station counts, showing collisions, delay and the throughput distribution per scheme.

7.5 Threats to Validity

Every marker in Figures 4–6 is a single, un-averaged stochastic run (random backoff, random p decisions, random placement), so small differences at one N should not be over-interpreted. In particular, in Figure 5 CSMA/CD does not exceed the randomised-deferral schemes in throughput at large N , although Table 3 and Eq. (6) predict that it should. Plausible factors are (i) the 10 ms sampling slot, which lets a collision persist up to one slot before it is noticed; (ii) CSMA/CD's 1-persistent sensing phase, which inherits the synchronised-restart problem that BEB only partly removes; and (iii) the small number of runs per point. Multi-seed averaging with confidence intervals is the natural next step.

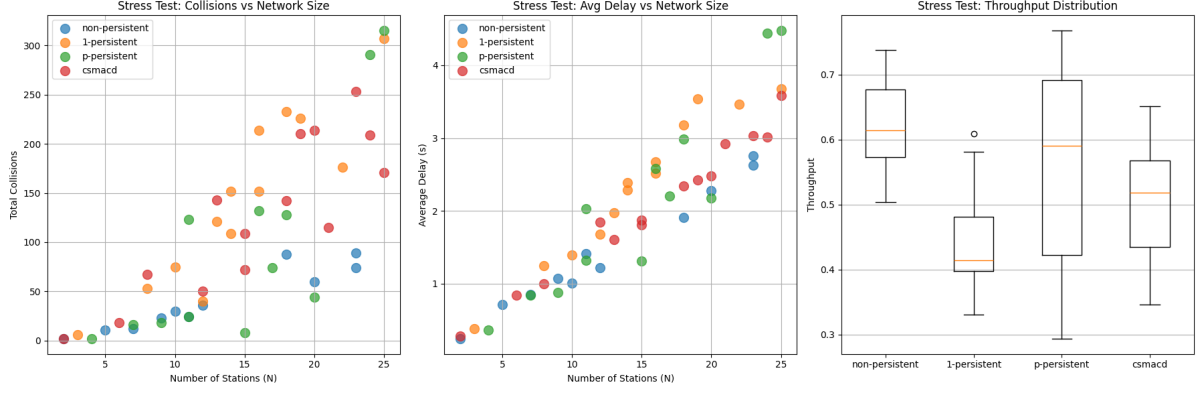


Figure 6. Stress test up to $N = 25$ stations: total collisions versus N (left), average delay versus N (middle) and box-plot of throughput per scheme (right). Above $N \approx 15$, 1-persistent, p -persistent and CSMA/CD each accumulate 170–315 collisions at $N = 25$ whereas non-persistent CSMA stays below about 90 collisions up to $N \approx 23$. Average delay grows roughly linearly with N for all schemes, reaching 3.6–4.5 s at $N = 25$ for the aggressive schemes. Median throughput is highest for non-persistent CSMA (about 0.61) and lowest for 1-persistent (about 0.41); p -persistent has the widest inter-quartile range, showing high sensitivity to the choice of p relative to N .

8 Key Analytical Findings & Comparative Discussion

8.1 Greedy versus Random Deferral

The 1-persistent scheme is greedy: every station that becomes ready during a busy period transmits with probability 1 when the medium is released. As N grows, the expected number of waiting stations at that instant grows with it, and collisions become nearly certain. This is the mechanism behind the collapse seen in Table 3 (throughput falls from 0.894 at $N = 2$ to 0.284 at $N = 20$, with delay rising from 0.115 s to 0.985 s) and in Figures 5 and 6. Non-persistent and p -persistent CSMA break the synchronisation by deferring for a random time or by a random slot decision, and so preserve stability. The price is idle time: a non-persistent station may sleep although the channel has just become free.

8.2 Impact of Early Abort

Early abort changes the cost of a collision. In classical CSMA a collision at 1 ms wastes the remaining airtime of a 100 ms frame. In CSMA/CD the wasted time is bounded by about 2τ , below 20 ms in this testbed. Table 3 shows the resulting benefit: wasted collision time falls from 100 ms to under 20 ms, and at $N = 20$ CSMA/CD attains 0.792 throughput against 0.612 for non-persistent and 0.284 for 1-persistent, i.e. about 180% higher utilisation than 1-persistent. The raw single-run results of Figure 5 do not reproduce this ordering (Section 7, Threats to Validity), so the size of the benefit should be regarded as configuration- and sample-dependent until averaged over more runs.

8.3 Distance versus Collision

A station learns of a transmission only after the signal reaches it, so the interval τ after any transmission begins is a blind period. Collision probability therefore scales with $a = \tau/T_{tx}$: a longer bus or a shorter frame increases collisions for identical traffic. Propagation delay is the root cause of carrier-sensing failure, and it also sets the minimum-frame-size rule of Eq. (7).

9 Engineering Challenges & Technical Solutions

Thread-safe shared channel state.

The active-transmission dictionary is accessed from several UDP handler threads; all access is wrapped in `threading.Lock()` (Listing 1) to keep the “signals reached” count consistent.

Non-blocking UDP server shutdown.

A blocking `recvfrom()` cannot notice a stop request; `sock.settimeout(0.5)` wakes the loop twice a second to check a stop flag and exit cleanly.

Slotted time sampling in Python.

A station cannot transmit and receive truly concurrently, so transmission and sensing are interleaved in 10 ms slots, a discrete-event approximation of listen-while-talk whose accuracy is bounded by the slot length.

10 Conclusion

10.1 Conclusion

A distributed shared-medium emulator with physical propagation delay was used to compare 1-persistent, non-persistent, p -persistent and CSMA/CD access. The results support the main analytical predictions. Greedy 1-persistent access is unstable under load; randomising the deferral (non-persistent, or p -persistent with a suitable p) restores stability; and the persistence probability has a clear interior optimum, found at $p = 0.3$ for $N = 10$. Detecting collisions while transmitting and aborting early bounds the cost of each collision at about 2τ and, in the benchmark table, gives CSMA/CD the highest throughput at every N . The raw single-run plots show the same qualitative separation of 1-persistent from the rest, but not a clear CSMA/CD advantage, which motivates repeated-seed experiments as future work.

10.2 Glossary of Acronyms

Table 4. Acronyms used in this report.

Acronym	Expansion	Acronym	Expansion
BEB	Binary Exponential Backoff	MAC	Medium Access Control
CD	Collision Detection	NIC	Network Interface Card
CRC	Cyclic Redundancy Check	UDP	User Datagram Protocol
CSMA	Carrier Sense Multiple Access	IPC	Inter-Process Communication
FFI	Foreign Function Interface		

10.3 Declaration of Generative AI Usage

Generative AI tools (Anthropic Claude and Google Antigravity) were utilized as assistive aids solely for **LaTeX typesetting, TikZ diagram drafting, and Matplotlib plotting scripts** in `visualize.py`. All core MAC protocol logic, spatial propagation modeling in `channel.py`, socket IPC, and experimental validation were independently developed, executed, and verified by the student.